

REPORTE TAREA 2

ALGORITMOS Y COMPLEJIDAD

«¿Quién ganará? Greedy vs Dinámico vs Bruto.»

Rodrigo Cáceres

24 de junio de 2026

01:10

1. Resumen

En esta tarea se estudió un problema de selección de capítulos de anime bajo restricciones de minutos y energía, donde para cada anime solo es válido escoger un prefijo de capítulos. Se implementaron cuatro estrategias: fuerza bruta, programación dinámica y dos heurísticas greedy. El objetivo fue comparar correctitud, tiempo de ejecución, memoria utilizada y calidad de solución entre los distintos enfoques.

Para evaluar las implementaciones se generaron automáticamente casos de prueba pequeños y medianos, además de instancias controladas para aislar el efecto de la cantidad de animes, la cantidad total de capítulos, los minutos disponibles y la energía disponible. A partir de estas instancias se midieron tiempo y memoria, y se utilizó programación dinámica como referencia óptima para evaluar la calidad de los algoritmos greedy.

Los resultados muestran que la programación dinámica entrega la solución óptima, pero con mayor costo computacional; fuerza bruta solo es útil en casos pequeños; y las heurísticas greedy son mucho más rápidas y eficientes en memoria, aunque con una calidad inferior a la óptima. Entre ellas, la variante basada en energía tendió a acercarse más a la solución óptima que la basada en minutos.

2. Entorno experimental

Los experimentos fueron realizados en un computador con sistema operativo Windows 10 Education, procesador AMD Ryzen 5 2600 Six-Core, memoria RAM de 16 GB, velocidad de 2400 MHz y utilizando el compilador g++ en su versión 15.2.0. El desarrollo y la ejecución de los programas se hicieron en VSCode.

Para las mediciones se utilizaron los casos de prueba generados con un script de python siguiendo el formato descrito en el enunciado. En particular, se consideraron instancias pequeñas para validar correctitud y para estudiar el comportamiento de los algoritmos al variar la cantidad de animes, la cantidad

de capítulos, los minutos disponibles y la energía disponible.

Los archivos de entrada, salida y medición fueron generados por los scripts del proyecto, de modo que los resultados presentados puedan reproducirse a partir del código fuente y de los casos de prueba correspondientes.

3. Resultados y Análisis

En esta sección se presentan los resultados experimentales obtenidos al ejecutar las cuatro implementaciones sobre los casos de prueba generados automáticamente. Los casos pequeños del conjunto `testcases` se utilizaron para validar correctitud y comparar fuerza bruta con programación dinámica; además, se construyeron casos de prueba para aislar la cantidad de animes, la cantidad total de capítulos, los minutos disponibles y la energía disponible.

3.1. Síntesis global

La Tabla 1 resume el comportamiento promedio observado en las ejecuciones realizadas. La programación dinámica es claramente más costosa en tiempo y memoria que los métodos greedy, pero es la única implementación exacta; por ello se usa como referencia para evaluar la calidad de las diferentes implementaciones greedy.

Algoritmo	Tiempo promedio (ms)	Memoria promedio (KB)
Fuerza bruta	38.44	8444.44
Programación dinámica	168.97	12416.23
Greedy 1	33.63	8372.46
Greedy 2	34.20	8370.51

Cuadro 1: Resumen global de tiempo y memoria. Los valores son promedios sobre los casos efectivamente ejecutados por cada algoritmo.

3.2. Tiempo de ejecución

La Figura 1 muestra el tiempo promedio según la cantidad de animes. Se observa que la programación dinámica crece de manera mucho más pronunciada que los métodos greedy, mientras que estos últimos se mantienen cercanos a una línea constante pero no plana. Esto concuerda con la diferencia esperada entre un algoritmo exacto que explora un espacio de estados más grande y variantes greedy que toman decisiones locales.

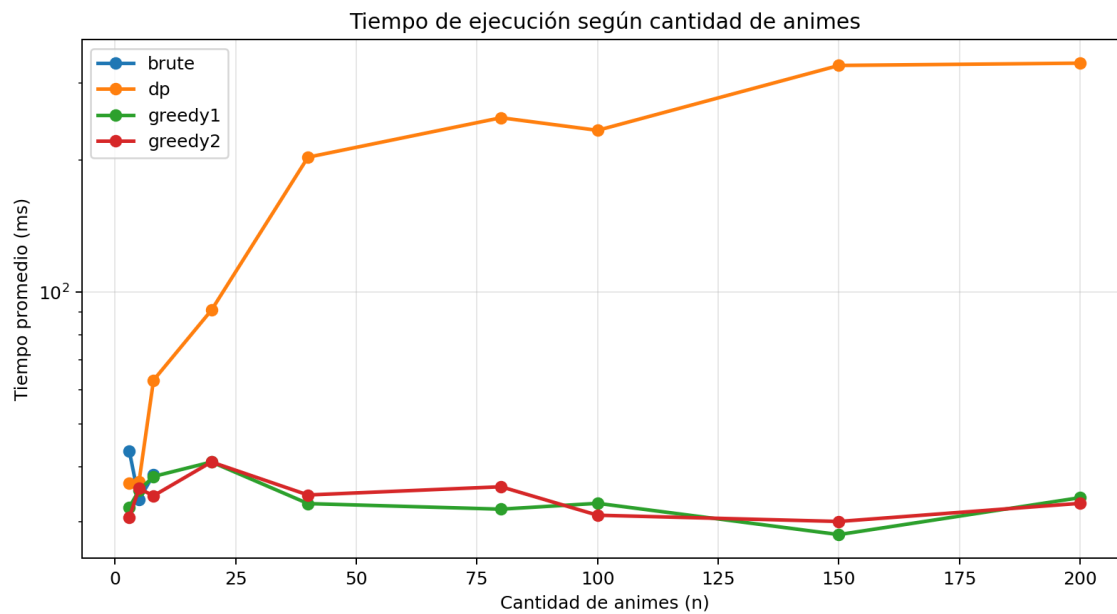


Figura 1: Tiempo de ejecución según cantidad de animes. La escala vertical está en logaritmo para distinguir mejor la separación entre los métodos.

La Figura 2 presenta el tiempo según la cantidad total de capítulos. Aquí la dispersión es mayor porque esta variable no se aisló por completo, o sea, cada caso pertenece a una 'familia' distinta y, además de variar el número de capítulos, también pueden cambiar el número de animes, los presupuestos disponibles y la distribución interna de las duraciones y energías. Aun así, se mantiene la tendencia general de que la programación dinámica es el método más costoso, mientras que los greedy y la fuerza bruta en casos pequeños son mucho más livianos.

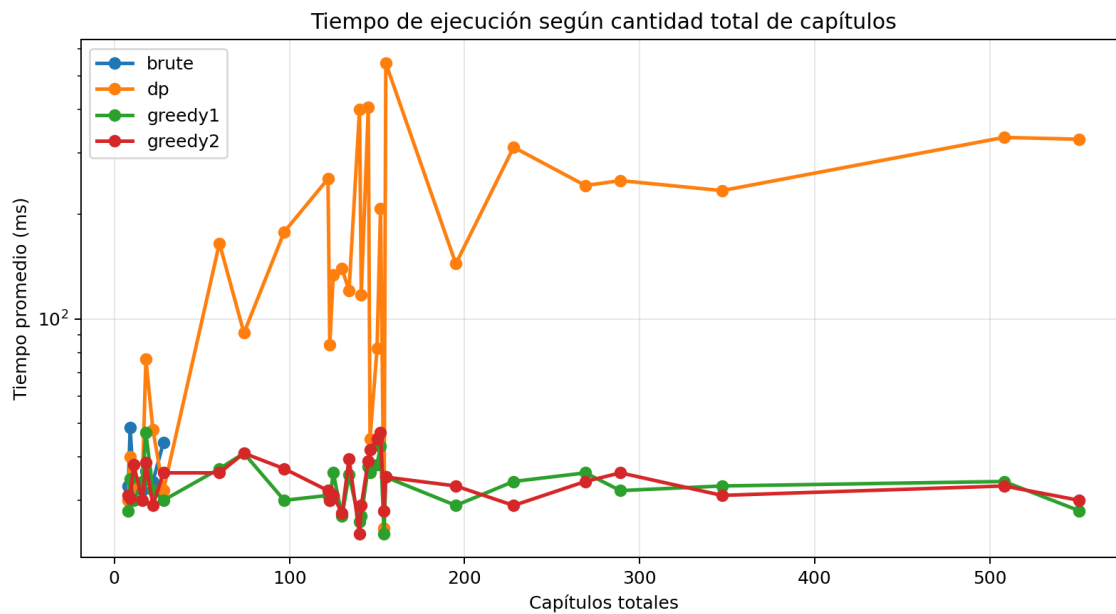


Figura 2: Tiempo de ejecución según cantidad total de capítulos.

Las Figuras 3 y 4 muestran el comportamiento respecto de los presupuestos de minutos y energía, respectivamente. En ambos casos la programación dinámica sigue siendo la que más se resiente cuando el espacio de estados crece, mientras que los greedy apenas cambian. Las pequeñas oscilaciones en las curvas no son una anomalía ya que provienen de que cada punto corresponde a una instancia distinta y por lo tanto el costo real depende también de la estructura interna de los animes y no solo del valor del presupuesto.

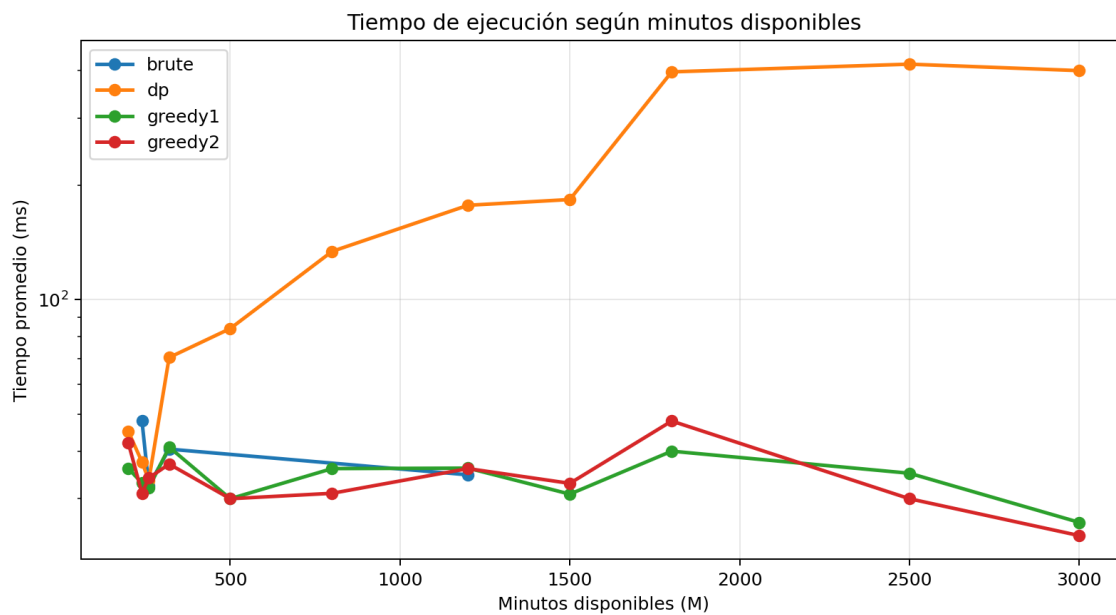


Figura 3: Tiempo de ejecución según minutos disponibles.

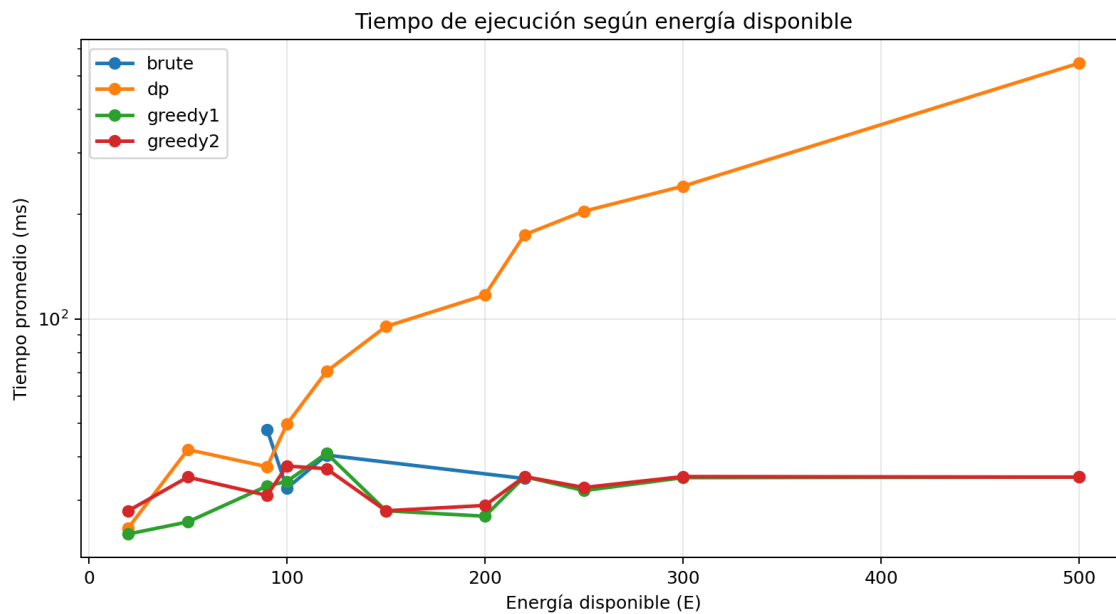


Figura 4: Tiempo de ejecución según energía disponible.

3.3. Uso de memoria

La memoria presenta un patrón más estable que el tiempo, aunque también exhibe variaciones entre casos. En los gráficos, los métodos greedy se mantienen cerca de un valor base muy similar entre sí, lo que es esperable porque su lógica solo requiere estructuras pequeñas para ordenar y evaluar opciones. Por otro lado, la programación dinámica utiliza tablas de estados y, por ello, su consumo de memoria es mucho mayor.

La Figura 5 resume el uso de memoria según la cantidad de animes. Se aprecia que los greedy prácticamente permanecen en un nivel plano, mientras que programación dinámica incrementa su consumo a medida que crece la cantidad de animes. Esto es coherente con el hecho de que la programación dinámica necesita almacenar información adicional para combinar subproblemas.

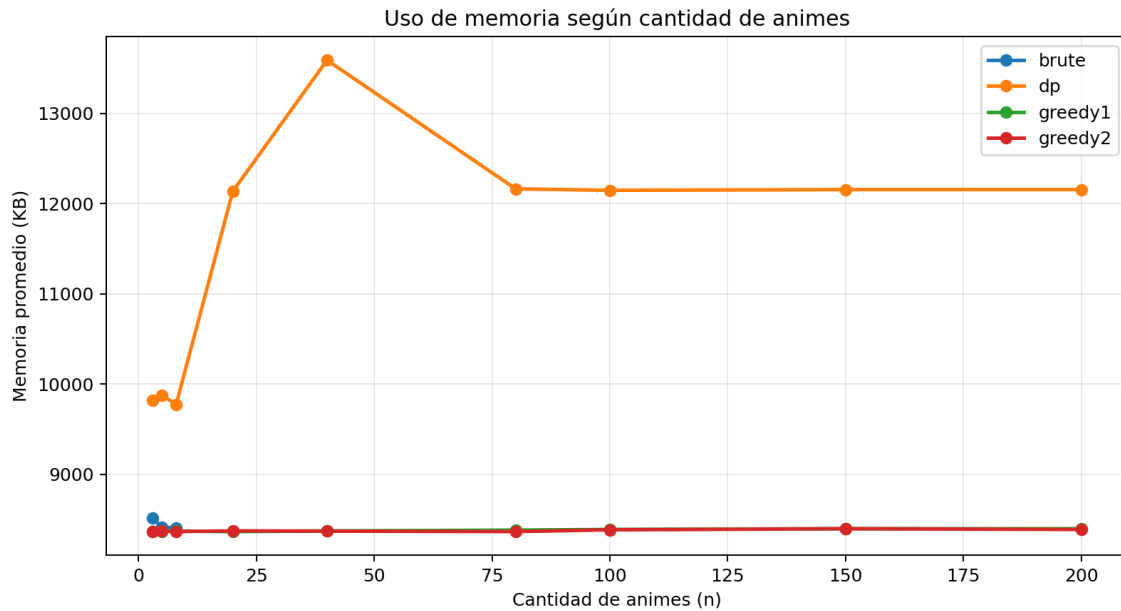


Figura 5: Uso de memoria según cantidad de animes.

Las Figuras 6, 7 y 8 muestran que la memoria no varía de forma estrictamente monótona para la implementación de programación dinámica. Esto no necesariamente indica un error pues el sistema operativo puede afectar a muchas mediciones como el runtime, la reserva interna de memoria, la distribución concreta de cada instancia, etc. El punto es que las curvas deberían mirarse como una tendencia mas que como un función exacta de lo medido.

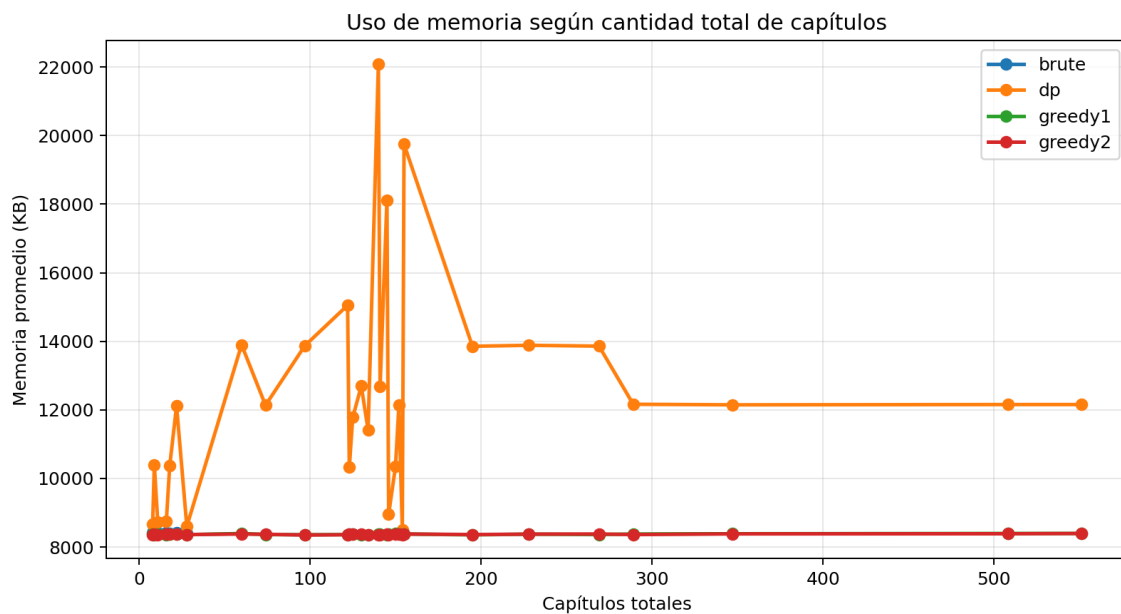


Figura 6: Uso de memoria según cantidad total de capítulos.

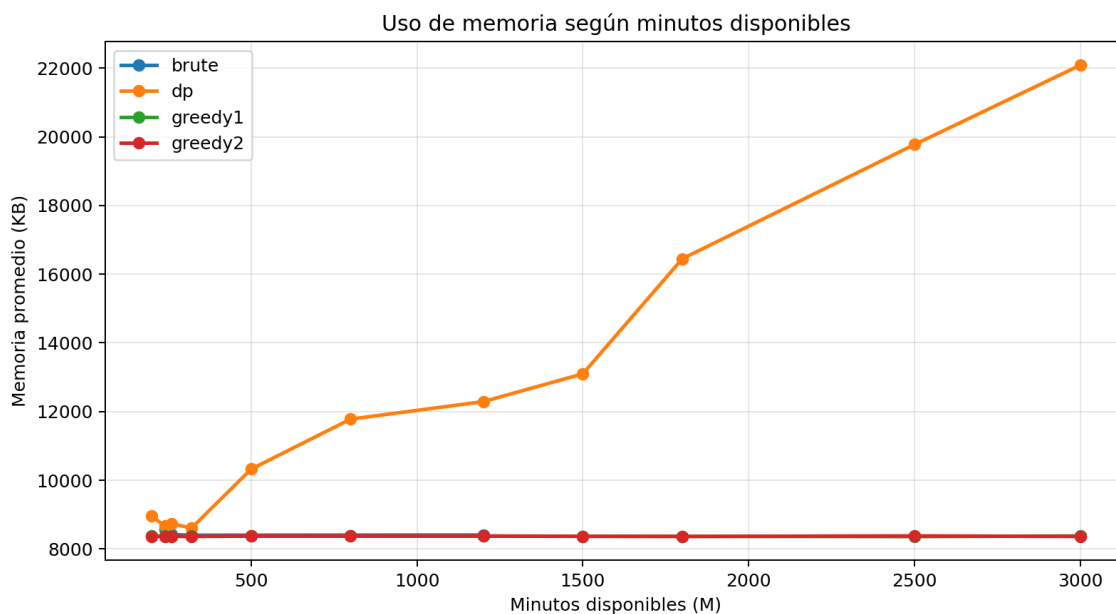


Figura 7: Uso de memoria según minutos disponibles.

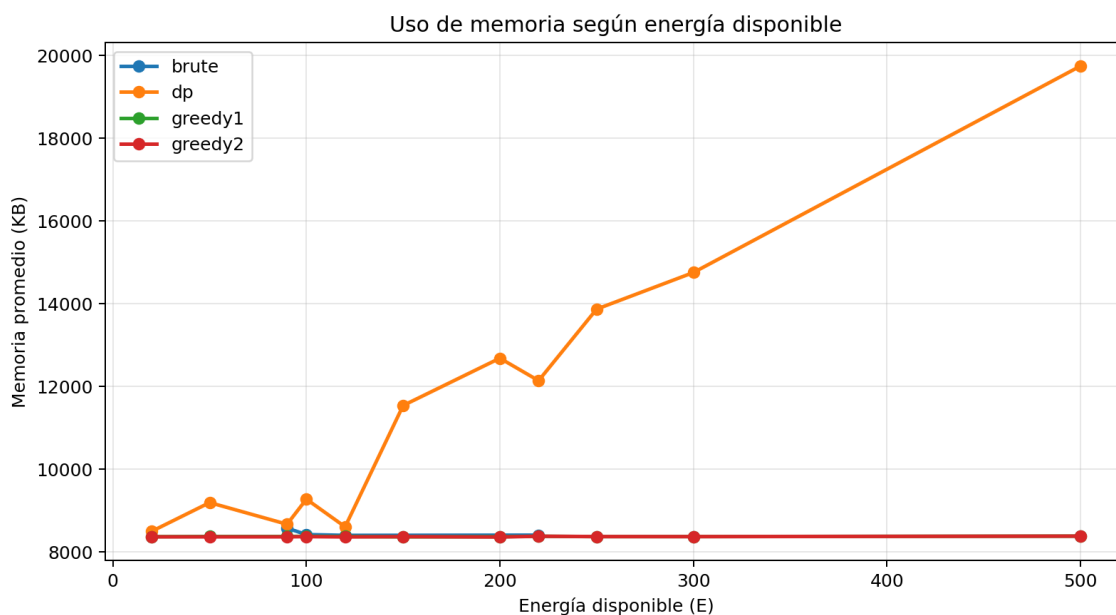


Figura 8: Uso de memoria según energía disponible.

3.4. Comparación de calidad respecto del óptimo

La Figura 9 compara la solución de los métodos greedy con el óptimo entregado por programación dinámica, usando la razón

$$\text{calidad} = \frac{\text{valor greedy}}{\text{valor optimo}}$$

Como la referencia óptima proviene de la programación dinámica, el valor 1 representa igualdad con la mejor solución encontrada. Se observa que greedy2 suele mantenerse más cerca de 1 que greedy1, especialmente en instancias medianas y grandes, mientras que greedy1 pierde calidad con mayor rapidez. Esto sugiere que el criterio basado en satisfacción por energía es, en este conjunto de pruebas, una aproximación más robusta que el criterio basado en satisfacción por minuto.

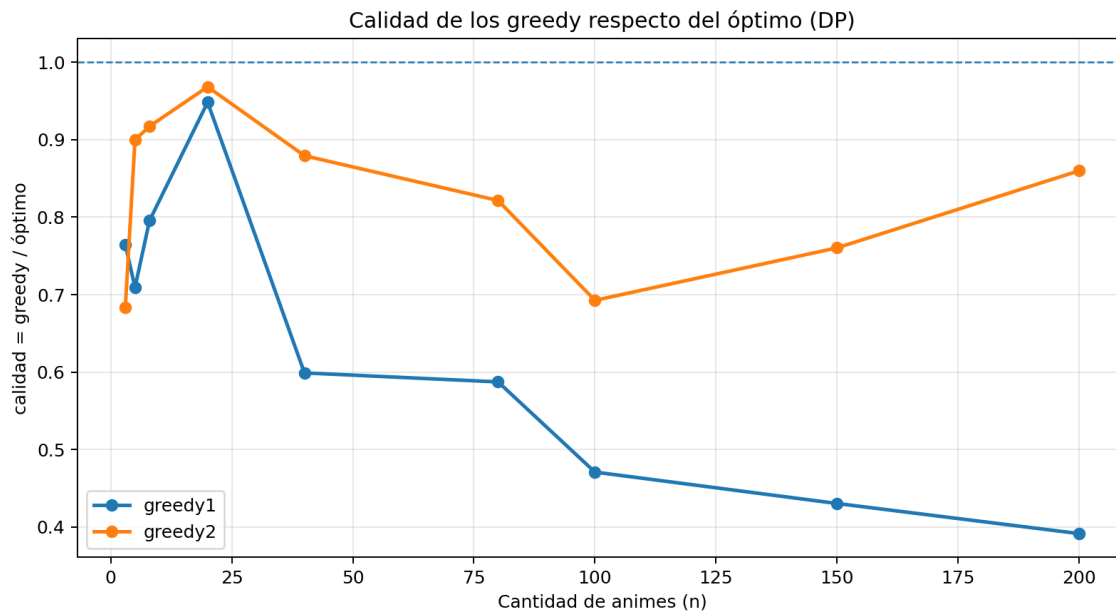


Figura 9: Calidad de solución de los greedy respecto del óptimo entregado por programación dinámica.

3.5. Comentarios finales sobre las mediciones

La fuerza bruta solo se ejecutó en instancias pequeñas, porque su costo crece muy rápido y no es razonable exigirle los casos medianos y grandes. En cambio, programación dinámica entrega la referencia exacta para comparar correctitud y calidad, mientras que los métodos greedy permiten estudiar el trade-off entre rapidez y precisión. A fin de cuentas, las mediciones muestran los siguientes comportamientos: exactitud a mayor costo para DP, y rapidez con menor calidad para las variantes greedy.

4. Conclusiones

En esta tarea se implementaron cuatro estrategias para resolver el problema de selección de capítulos por anime bajo restricciones de minutos y energía: fuerza bruta, programación dinámica y dos variantes greedy (greedy 2: satisfacción por energía; greedy 1: satisfacción por minuto). A partir de los experimentos realizados, se observó que la programación dinámica entrega la solución óptima en los casos evaluados, pero a costa de un mayor consumo de tiempo y memoria que los métodos greedy.

La fuerza bruta permitió verificar la correctitud de las otras implementaciones en instancias pequeñas, pero su costo computacional crece rápidamente, por lo que deja de ser práctica para tamaños media-

nos o grandes. Por su parte, los algoritmos greedy resultaron mucho más rápidos y livianos en memoria, aunque con una calidad de solución inferior a la óptima. Entre ambos, la variante basada en satisfacción por energía mostró un comportamiento más cercano al óptimo que la basada en satisfacción por minuto.

En síntesis, los resultados confirman el trade-off esperado entre exactitud y eficiencia. La programación dinámica es la alternativa adecuada cuando se requiere la mejor solución posible, mientras que las heurísticas greedy son útiles cuando se prioriza rapidez y bajo consumo de recursos. La fuerza bruta cumple un rol principalmente de validación en casos pequeños.

5. Implementaciones

El código fuente completo de la tarea se encuentra disponible en el siguiente repositorio de GitHub:
<https://github.com/RodrigoalDelPlanetaGol/Tarea-2-Algoritmos-y-complejidad>